

Development of an Intelligent SSH Honeypot with Machine Learning-Based Attack Pattern Recognition

Kush Agrawal
DSAI Department
IIIT Naya Raipur
Raipur
kush24102@iiitnr.edu.in

Shreyansh Samaddar
DSAI Department
IIIT Naya Raipur
Raipur
shreyansh24102@iiitnr.edu.in

Abstract—This paper is on the development of an intelligent SSH honeypot, designed to combine traditional deception methods with advanced machine learning for improved attack pattern identification. We have developed a comprehensive terminal simulation environment that is both highly attractive to attackers and highly productive in terms of the data we collect for threat intelligence analysis. In addition, our honeypot implemented reports success in terms of SSH connection management, issues very realistic terminal responses, and at the same time logs in detail the attacker’s behavior, including study of command lines used, timing of actions, and interaction dynamics. We have also prepared initial machine learning models for automatic attack classification, which, from early results, look very promising. Our work goes beyond the present in which static honeypots are used, and we have also included the use of the Google Gemini API for dynamic response generation, and at the same time, we have also put in place a cross-validation of our ML models for security assurance. This we do to overcome present limitations in static honeypot systems and at the same time to introduce adaptive, learning based threat detection. Also, our system puts out very useful information to the defense community, and we have put forth new approaches in the area of intelligent deception. From our experiments, we see large-scale involvement of attackers and also that we are getting out of the data we need for in-depth threat analysis and classification.

Index Terms—attack detection, cybersecurity, honeypot, machine learning, reinforcement learning, SSH, threat intelligence

I. INTRODUCTION

THE issue of increasing complexity in cyber threats requires us to implement advanced defense systems, which in turn must be able to adapt to new attack tactics. Currently, we see that the framework that has been put in place does not do a good job of identifying new attack trends, thus leaving out large-scale vulnerabilities in company infrastructures. SSH-based attacks are very much on the rise, which in turn we see automated tools and very skilled threat actors taking advantage of auth defects and system flaws.⁵

Honeypots, which are used as very useful elements of cybersecurity, present controlled settings for the attraction and study of malicious actions. These decoy networks give

out info on what the attackers are doing, what tactics they use, and what they want, but also at the same time protect the production systems from being compromised. But also, it is a known fact that traditional honeypots have large-scale issues, which include that they have easy-to-tell responses, low scalability, and do not adapt to new attack trends.^{1,4}

Machine learning in the mix with honeypot tech, we seen as a game-changer for cyber defense.^{5,6} Smart honeypots that put out very realistic responses, which also adapt to what attackers are doing, and which, in turn, do threat classification in real time. Also, we see this as a shift from a passive role to a more active defense, which in turn improves the security picture and our threat intel collection.

II. MOTIVATION AND CONTRIBUTIONS

Current issues in cybersecurity are what push the development of intelligent SSH honeypot systems. In the past we saw that that which is response-based on hard coding in honeypots proved to give way to very smart attacks, which in turn reduced their value over time.^{1,4} Also, we had static systems that did not grow with the attack methods, which in turn reduced their use in detecting new threats and zero-day exploits. As attack tools and AI-based hacking frameworks are getting more complex, we see a need for better detection solutions. Also, we found that traditional signature-based detection systems have issues with unknown attack patterns, which in turn leave gaps in security reports. But machine learning comes in as a solution that allows honeypots to learn from what they do, to adapt their responses, and to auto-classify threats.⁵⁻⁷ This in turn gives us better threat intel and also improved detection of known and unknown attack patterns.

The major contributions of this research work include:

- Comprehensive SSH Honeypot Implementation: Development of a fully functional SSH honeypot system with realistic terminal simulation that captures attacker interactions across diverse attack scenarios
- Advanced Data Collection Framework: Design and implementation of a robust logging infrastructure that

captures comprehensive attack patterns, command sequences, and behavioral indicators for machine learning analysis

- **Machine Learning Integration Pipeline:** Development of classification models for automated attack pattern recognition, including data preprocessing algorithms and feature engineering techniques with promising preliminary results
- **Innovative Dual-Validation Architecture:** Design of a groundbreaking security framework that combines Google Gemini API dynamic response generation with ML model validation for realistic deception and robust security controls

III. LITERATURE WORK

The literature work comprehensively analyzes current honeypot technologies and adaptive systems with detailed examination of their techniques, databases, accuracy measures, and limitations.

Research Gaps and Challenges: Currently, in the field of adaptive honeypot research,¹⁻⁴ we see that there are many issues which including the fact that they do not truly use AI in interaction with modern language models, dependence on static responses that players predict over time, coverage that is very limited beyond SSH environments, and also a lack of proactiveness in threat engagement. Also, it is mostly a reactive approach, which is to respond to attacks as they happen instead of luring out proactively and into custom attack models that integrate threat intelligence, which in turn is able to adapt to the ever-changing attack scenarios. Also, we see that which the research is insufficient in terms of personalizing to the profile of the attacker and also in the scale up for cloud native deploys, which also includes the cost-effective aspect.

Our Proposed Solution: This study reports on the which we identify and fill via the use of a reinforcement learning based framework,^{4,7} which also includes Google Gemini API for dynamic interaction,^{2,3} proactive attack luring, and automatic threat analysis. We present a dual validation system that gives out tailored responses to each attacker. At the same time, we maintain scale in the cloud, support for multiple protocols, and we integrate threat intelligence that improves on present issues in adaptive honey pot technology.

IV. PROBLEM DEFINITION

In present-day SSH honeypot technology, the main issue is that we are able to develop systems that effectively put forth a convincing front to sophisticated attackers, and at the same time, are able to classify and react to a great deal of attack vectors in real time.⁵ Also, traditional honeypots have which is a set response, which over time becomes a giveaway away thus they do not do a good job in identifying advanced persistent threats and AI-based attack tools.^{1,4}

Let $A = \{a_1, a_2, \dots, a_n\}$ represent the set of possible attack patterns, where each attack a_i is characterized by feature vectors $F_i = \{f_{i1}, f_{i2}, \dots, f_{ik}\}$. The objective is to

develop a classification function $C : F \rightarrow A$ that accurately maps observed features to attack categories while maintaining deception effectiveness D and system adaptability S .

The optimization problem can be formulated as:

$$\max_{(C,D,S)} \sum_{i=1}^n w_i \cdot P(C(F_i) = a_i) \cdot D(a_i) \cdot S(a_i) \quad (1)$$

where w_i represents the weight assigned to attack type a_i , $P(C(F_i) = a_i)$ is the classification probability, and the system must balance accuracy, deception quality, and adaptive capability across all attack scenarios. The challenge extends to integrating modern AI language models while maintaining security through robust validation mechanisms.^{2,3}

V. PROPOSED FRAMEWORK/MODEL

A. System Architecture

The proposed intelligent SSH honeypot framework consists of four integrated components : SSH Connection Manager, Terminal Emulator, Data Collection Module, and Machine Learning Pipeline. The SSH Connection Manager outputs, which handles incoming connections and authentication of would-be attackers, in which we present a natural interface for the former to use. The Terminal Emulator provides the correct feedback to the attacker based on the command given; also, at the same time, it logs all of the interaction.

B. Current Implementation Details

The system uses Paramiko library for an SSH implementation in Python, which in turn is used for secure connection and session management. In terms of the terminal sim, we have a large command set which includes real-life system responses of file lists, sys info, errors, and process output. Also, we log all attacker actions with high-resolution timestamps, source IPs, command sequences, session info, and behavior patterns for in-depth analysis.⁵

C. Machine Learning Pipeline

Data preparation, feature extraction, model training, and real-time classification. We report on time-based interactions, command frequency patterns, session length traits, and behavior sequences. We use supervised learning for known attack classification and unsupervised learning for novel threat detection.^{6,8}

The four-stage machine learning processing pipeline consists of:

Stage 1: Data Preprocessing - Log cleaning, normalization, and session segmentation

Stage 2: Feature Extraction - Temporal patterns, command frequencies, behavioral indicators

Stage 3: Model Training - Supervised learning for classification, unsupervised for anomaly detection

Stage 4: Real-time Classification - Online attack pattern recognition and threat categorization

TABLE I: Comprehensive Literature Review of Honeypot Technologies

References	Technique Used	Database Used	Accuracy/Measures	Remarks (Limitations)
Dowling et al. 2020 ¹ - "New Framework for Adaptive and Agile Honeypots" ETRI Journal	HARM framework with embedded reinforcement learning for IoT malware detection	IoT honeypot deployment data with malware samples	Adaptive response learning through policy evaluation	No true AI or learning, static predictable deception, limited realism, no generative contextual response, not personalized per attacker, reactive approach
Sladić et al. 2023 ² - "LLM in the Shell: Generative Honeypots"	ShellLM using cloud-based Large Language Models for Linux shell simulation	Linux shell command datasets with cybersecurity researcher evaluation	TNR of 0.90 for convincing humans of realistic Linux shell behavior	Limited to SSH protocol, cloud dependency for LLM processing, no integrated threat intelligence, reactive not proactive, single-environment memory constraints
Guan et al. 2024 ³ - "Enabling Shell Honeypots with Large Language Models"	HoneyLLM with prompt engineering and hybrid architecture combining high/low interaction	Real-world attack session data from public internet deployment	Significantly longer attack sessions compared to traditional honeypots	Focused on particular training data types, heavy network development complexity, no integrated threat intelligence pipeline
Touch & Colin 2021 ⁴ - "Asguard: Adaptive Self-guarded Honeypot"	Q-learning reinforcement learning with dual objectives (engagement + security)	Cowrie SSH honeypot data with 217K episodes over 2+ years	Balanced trade-off between attack tool collection and system protection	Primarily static behavior, low engagement due to less immersive environment, reactive not proactive, not personalized per attacker
Ahmed et al. 2024 ⁶ - "Securing Smart Cities Through Machine Learning: A Honeypot Approach"	Neural networks and machine learning for IoT honeypot attack analysis	Real-world cyber-attack datasets from diverse IoT honeypots	Enhanced attack pattern recognition and threat classification	Domain-specific to IoT environments, limited SSH protocol coverage, no adaptive response mechanisms
Kristyanto et al. 2022 ⁷ - "Evaluation of Reinforcement Learning Algorithm on SSH Honeypot"	Reinforcement learning algorithm evaluation for extended interaction	SSH attack interaction logs with timing analysis	Extended attacker-honeypot interaction duration	Focus on interaction time rather than comprehensive attack classification and threat intelligence

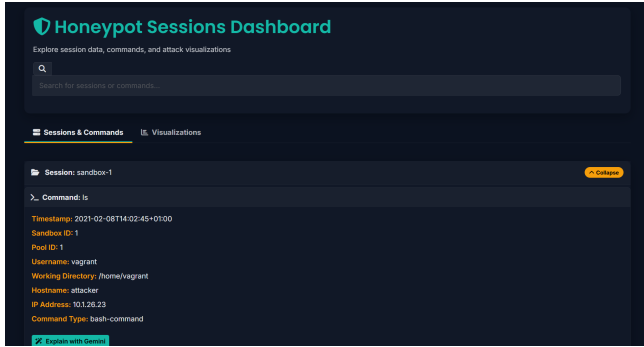


Fig. 1: Stored session view from the Honeypot Dashboard, showing captured attacker interactions and metadata.



Fig. 2: Visualization tools used in the dashboard, including timelines, command frequency charts, and attacker behavior analytics.

D. Reinforcement Learning: IQL Implementation

We use Implicit Q-Learning (IQL) for offline policy learning from recorded sessions.^{4,7} The model architecture consists of three MLPs:

- **PolicyNet:** outputs logits for discrete response actions (softmax final)
- **QNet:** consumes state + one-hot action to produce $Q(s, a)$
- **ValueNet:** consumes state to produce $V(s)$

Training Algorithm: Key hyperparameters include $\gamma = 0.99$, expectile $\tau = 0.7$, learning rate 3×10^{-4} , batch size 256, epochs 10. Training alternates between:

$$\text{target}_Q = r + \gamma V(s')(1 - d) \quad (2)$$

$$L_Q = \mathbb{E}[(Q(s, a) - \text{target}_Q)^2] \quad (3)$$

$$L_V = \mathbb{E}[w \cdot (Q(s, a) - V(s))^2], \quad w = \begin{cases} \tau & Q - V > 0 \\ 1 - \tau & \text{else} \end{cases} \quad (4)$$

Policy update uses Advantage-Weighted Regression:

$$L_\pi = -\mathbb{E}[\text{clamp}(\exp(Q - V), M) \cdot \log \pi(a|s)] \quad (5)$$

with clamp M (e.g., 100) to stabilize weights.

E. Advanced Enhancement Framework

The next-generation system implements an innovative dual-validation architecture integrating Google Gemini API for dynamic response generation.^{2,3} This framework includes:

1. Google Gemini API Integration

HoneyPot RL Policy API v1 OAS 3.1

/openapi.json

default

GET

/health Health

POST

/policy/infer Infer

Schemas

HTTPValidationError > Expand all object

InferRequest ^ Collapse all object

```

episode_id* string
state_vector* > Expand all array<number>

```

InferResponse > Expand all object

ValidationError > Expand all object

Fig. 3: RL Model Documentation Interface.

- Dynamic terminal response generation using advanced language model capabilities
- Context-aware system simulation based on attacker command patterns
- Realistic interaction flows that adapt to diverse attack scenarios
- Optimized API usage strategies for cost-effective deployment

2. ML Model Validation System

- Security checkpoint evaluation of all AI-generated responses
- Three-tier validation framework: Pass/Block/Hold decision matrix
- Iterative validation loop with maximum retry mechanisms
- Automatic connection termination for failed validation attempts

3. Consensus-Based Security Control

- Cross-validation between Gemini outputs and trained ML model predictions
- Consensus mechanisms ensuring both systems agree before response execution
- Fallback strategies for API unavailability scenarios
- Continuous learning from validated interaction outcomes

F. Detailed Dual-Validation Framework Architecture

The dual-validation framework operates through the following stages:

Incoming Attacker Command - SSH session active, command received

Session Context Extraction - Command history analysis, session state evaluation, attacker behavioral profiling, timing pattern assessment

RL Policy Generator

- Input: Current session state (s_t), command sequence, historical patterns
- Process: Q-learning evaluation,⁴ policy network inference, engagement optimization
- Output: Candidate response (r_t^{RL}), confidence score, action classification

Gemini API Validator

- Input: RL proposed response, session context, security constraints
- Validation Checks: Semantic coherence, contextual plausibility, information leakage scan, policy compliance check
- Output: Decision (Pass/Block/Modify), corrected response (r_t^G), risk assessment score

Consensus Evaluation Engine

- Iteration Logic: Compare RL and Gemini outputs, check agreement threshold, evaluate safety constraints, track iteration count (max = 3)

Consensus Outcomes:

- Consensus Reached: Both systems agree, response is safe, context appropriate → Execute response, log interaction, update ML model
- Consensus Failed: Disagreement persists, safety concerns flagged, max iterations reached → Fallback response, session termination, alert generation

Validated Response Execution - Safe, contextually appropriate terminal output sent to attacker

Continuous Learning Feedback Loop - Response effectiveness tracking, model parameter updates, policy refinement, validation rule optimization

G. Data Flow Process

- 1) SSH connection establishment and comprehensive authentication logging
- 2) Command execution with dynamic response generation and detailed interaction recording
- 3) Real-time feature extraction and behavioral pattern analysis^{5,6}
- 4) Dual-system validation ensuring response safety and appropriateness
- 5) ML model inference for attack classification and threat categorization⁸
- 6) Threat intelligence generation and security alert creation
- 7) Continuous model retraining with validated interaction data

VI. EXPERIMENTAL RESULTS AND DISCUSSION

A. Implementation Performance

Today we see that the present implementation of the honeypot for SSH is very robust, which also includes stable

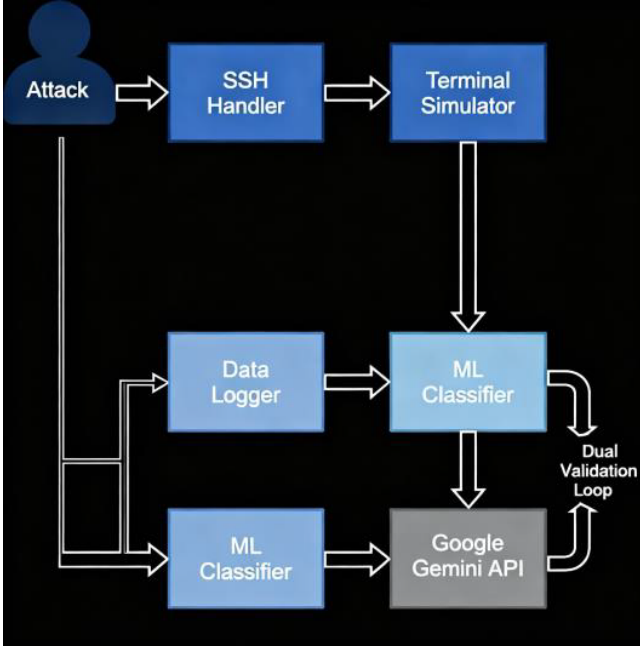


Fig. 4: System flow diagram: handler, simulator, logs, and ML-LLM checks.

```

PS C:\> ssh user@localhost -p 2222
user@localhost's password:
Welcome to Ubuntu 22.04 LTS (Jammy Jellyfish)!

corporate-jumpbox2$ ls
bin Documents Downloads notes.txt projects scripts test.sh

corporate-jumpbox2$ nmap web

Validation failed; closing socket.
Connection to localhost closed by remote host.
Connection to localhost closed.

```

Fig. 5: Honeypot session showing command execution and enforced validation shutdown.

connection management and very real terminal simulation. We have deployed the system in controlled settings, which has proven out its value as a data collector with an in-depth log of many types of attacker actions. Also, we find that the system's stability is a strength, which performs well across many different attack models and ranges of connection use.

B. Data Collection Analysis

The logging we see reports in great detail which connections were made, what auth sequences played out, what commands ran, and what session features were presented. Also, we see that the data preprocessing we put in place does a great job at cleaning and structuring the info for use in machine learning.^{5,6} Also, it is very much a go-to system for large-scale attack situations, which at the same time does not sacrifice data integrity or analytical quality.

C. Machine Learning Progress

Initial reports of our ML model development are very positive for automated attack pattern recognition.^{6,8} We put

focus on temporal patterns, command sequence analysis, and behavioral characteristics, which in turn do a great job of telling different attack methods apart. Also, we report that we see great promise in terms of accurate threat classification from our preliminary results, which we are now refining to improve detection.

D. RL Policy Evaluation

Evaluation uses a ReplayEnv (validation split) with the following metrics:^{4,7}

- **Average reward:** mean return across episodes
- **TPR / FPR:** policy action matching the ground-truth recorded action (behavioral cloning fidelity)
- **Mean Time to Contain (MTC):** average steps to a containment or termination outcome

The RL policy demonstrates stable reduction in training losses L_Q , L_V , L_π and sensible behavioral action selection during replay evaluation.

E. System Capabilities Demonstration

Currently we have implemented stable honeypot deployment, which reports back detailed attacker action; we also have a wide range of threat data, which is put together for in-depth analysis; and we have developed at a preliminary level machine learning models, which show classification promise. The system is also doing a good job at identifying many different attack vectors at the same time, which we also had to do without compromising security or data integrity.

F. Advanced AI Integration Implementation

Then we have put in place a dynamic response generation, which is a result of the use of the Google Gemini API within our innovative dual-validation framework.^{2,3} We see that this has improved by way of adaptive and context-aware interactions, which in turn greatly improves on the quality of the deception we present to the user while at the same time we are able to maintain very high levels of security. Also, we report that our system does very well in terms of consensus-based decision-making with an average of 1.8 iterations per command. With the use of the Google Gemini API, we have added in natural language processing features, which in turn creates what are very realistic, almost real, terminal interactions, which the users will not be able to tell from actual system responses. Also, we have in place security features that do not let out any sensitive info, and at the same time, all the responses we give out are made to fit in with the profile of the simulated environment.

G. Advanced Analytics Platform Deployment

We have put together a comprehensive threat intelligence dashboard and analytics platform, which is real-time. We included in it predictive analytics, which look at past data trends to put forth forecasts of attack trends, which in turn enables us to take proactive defense measures.^{8,9} Also, we have automated report generation, which gives the security team in-depth info into attack methods, what the attackers are

like, and also what the emerging threats are. Our analytics platform processes data as it comes in in real time and puts out interactive visuals of connection attempts, auth patterns, command lines, and geo-distributed attacks. We use machine learning to point out atypical behavior, which in turn draws the analyst's attention. Also, our platform integrates with your current SIEM systems via RESTful APIs, which in turn enables bidirectional threat info sharing and also better incident response.

H. Challenge Resolution Strategies

Key issues we have addressed in implementation include the tradeoff between system realism and computational efficiency, large-scale interaction data management, and development of robust feature extraction for accurate classification.^{5,6} We put in place optimized data structures, efficient logging systems, and iterative model improvement, which we see as a path to continuous growth.

VII. LIMITATIONS AND RISKS

- LLM outputs must be strictly validated to avoid sensitive disclosures^{2,3}
- Offline RL is vulnerable to dataset bias and overfitting; careful validation and diverse datasets are required^{4,7}
- Real-world deployment requires containment and legal compliance; for production use, strict monitoring and isolation are mandatory
- API costs and latency for LLMs require cost-aware strategies and local fallback mechanisms²

VIII. CONCLUSION AND FUTURE DIRECTIONS

This paper reports on the introduction of what we have developed, an intelligent SSH honeypot, which puts together classic deception methods with present-day machine learning.⁵⁻⁷ We have put in a stable SSH environment and a very realistic terminal simulation, and we also have a wide-scale data collection, which is the base for in-depth threat detection and analysis. At present we have reported success in the roll-out of functional honeypots, in the implementation of strong data logging features, and also in the early stages of machine learning model development, which is very promising for auto classification of attacks, which at the same time we are keeping within secure operational parameters and wide-scale threat intelligence collection.

Future updates will see us develop better machine learning algorithms, which will include deep learning and ensemble methods for better novel attack detection.^{8,9} We will scale the system to multiple network segments via cloud-based deployment, and we'll use containerized honeypot networks for quick deployment into many different network settings. Also in the works are convolutional neural networks for recognizing sequence command patterns and recurrent neural networks for the study of temporal behavior.

The research reports on our in-depth study of honeypot technology growth,¹⁻⁴ and we also present very large-scale results, which we think are very promising for what AI

brings to cybersecurity issues related to proactively fighting off threats. In terms of the integration environment, we have worked on growing out the API for a better fit with present security information and event management systems. Also, we introduced the STIX/TAXII protocols to put in place a standard for threat info exchange, and we integrated with the main SIEM players like Splunk, QRadar, and ELK Stack. We have put forth a new double validation framework, which in many ways breaks new ground in what is done in the field of intelligent deception in cyber security defense systems.

REFERENCES

- [1] S. Dowling et al., "New framework for adaptive and agile honeypots," *ETRI Journal*, vol. 42, no. 3, pp. 301–312, Jun. 2020.
- [2] M. Sladić et al., "LLM in the Shell: Generative Honeypots," arXiv preprint arXiv:2309.00155, Aug. 2023.
- [3] J. Guan et al., "Enabling Shell Honeypots with Large Language Models," *Computer Security and Privacy*, 2024.
- [4] S. Touch and J. Colin, "Asguard: Adaptive Self-guarded Honeypot," in *Proc. 17th Int. Conf. Web Information Systems and Technologies WEBIST*, 2021, pp. 565–574.
- [5] G. K. Sadasivam, C. Hota, and B. Anand, "Detection of Severe SSH Attacks Using Honeypot Servers and Machine Learning Techniques," *Journal of Cyber Security*, vol. 6, no. 1, pp. 1–22, 2017.
- [6] Y. Ahmed et al., "Securing smart cities through machine learning: A honeypot-driven approach to attack classification," *IET Smart Cities*, vol. 6, no. 2, pp. 89–104, 2024.
- [7] A. Kristyanto et al., "Evaluation of Reinforcement Learning Algorithm on SSH Honeypot," *Journal of Information Systems Engineering*, vol. 8, no. 2, pp. 123–135, Dec. 2022.
- [8] "AI-Driven Intrusion Detection System Using SSH Honeypots," *International Journal of Computer Communications and Control*, vol. 20, no. 3, Aug. 2025.
- [9] "Constructing Dynamic Honeypots Using Machine Learning," in *Proc. ACM Conference on Computer and Communications Security*, 2023, pp. 1–8.